

SQL Server Query Optimizer

Problem-Solving Scenarios (Interview)

Problem 1: Orders Page Becomes Slow After Data Growth

Scenario

You are working on an e-commerce system.

The **Orders** table recently grew to more than **10 million rows**.

A page that displays all orders from **year 2024** has become very slow.

Old Query

```
SELECT *  
FROM Orders  
WHERE YEAR(OrderDate) = 2024;
```

Problem 2: Customer Search by Phone Number

Scenario

The customer support team searches for customers by phone number.

The search works correctly but becomes extremely slow as the number of customers increases.

Old Query

```
SELECT CustomerId, Name, Phone  
FROM Customers  
WHERE Phone LIKE '%010%';
```

Problem 3: Slow Customer Orders Report

Scenario

A report displays all orders placed in **2024** by customers who live in **Egypt**.
The report was fast in development but slow in production.

Old Query

```
SELECT o.OrderId, o.OrderDate, c.Name
FROM Orders o
JOIN Customers c ON c.CustomerId = o.CustomerId
WHERE YEAR(o.OrderDate) = 2024
      AND c.Country = 'Egypt';
```

Problem 4: OR Condition Causing High CPU Usage

Scenario

You need to retrieve orders that either belong to a specific customer
or belong to customers from Egypt.
The query causes high CPU usage in production.

Old Query

```
SELECT o.OrderId, o.OrderDate, o.TotalAmount
FROM Orders o
JOIN Customers c ON c.CustomerId = o.CustomerId
WHERE o.CustomerId = 1200
      OR c.Country = 'Egypt';
```

Problem 5: Customers Who Have Orders

Scenario

You want to list all customers who have placed at least one order.
As the Orders table grows, performance degrades.

Old Query

```
SELECT *  
FROM Customers  
WHERE CustomerId IN (  
    SELECT CustomerId  
    FROM Orders  
);
```

Problem 6: Latest Orders Dashboard

Scenario

A dashboard shows the **latest 20 orders**.
As the Orders table grows, the page load time increases significantly.

Old Query

```
SELECT TOP 20 *  
FROM Orders  
ORDER BY OrderDate DESC;
```

Problem 7: Orders Count Per Customer (Yearly Report)

Scenario

A business analyst requests a report showing the number of orders
per customer for the year **2024**.

Old Query

```
SELECT CustomerId, COUNT(*) AS OrderCount
FROM Orders
WHERE YEAR(OrderDate) = 2024
GROUP BY CustomerId;
```

Problem 8: Filtering Using a Scalar Function

Scenario

A scalar function calculates the customer tier (Silver, Gold, Platinum).
This function is used in filtering, but performance is poor.

Old Query

```
SELECT c.CustomerId, dbo.GetCustomerTier(c.CustomerId) AS Tier
FROM Customers c
WHERE dbo.GetCustomerTier(c.CustomerId) = 'Gold';
```